Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 8

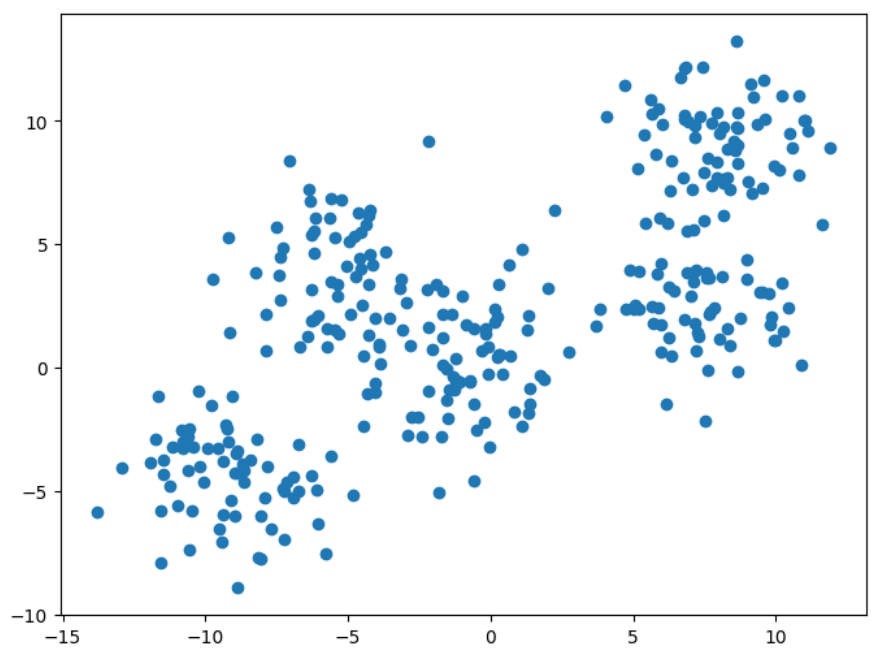
Cài đặt thuật toán Mahalanobis KMeans:

- Import các thư viện và tạo dữ liệu là các điểm như mẫu:

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(234)
x, _ = make_blobs(n_samples=330, centers=5, cluster_std=1.84)
plt.figure(figsize=(8, 6))
plt.scatter(x[:, 0], x[:, 1])
plt.show()
```

✓ 0.0s



- Dựa vào bài tập tuần 6 ta tạo các hàm để thực hiện thuật toán Mahalanobis_Kmeans:
- Hàm lấy ngẫu nhiên k điểm trung tâm của bộ dữ liệu:

```python
def initiate_centroids(k, dset):
    centroids = dset.sample(k)
    return centroids
```

✓ 0.0s

- Hàm tính khoảng cách mahalanobis:

```python
def mahalanobis_distance(x, mu, inv_cov):
    diff = x - mu
    dist_sq = np.dot(np.dot(diff, inv_cov), diff.T)
    return np.sqrt(dist_sq)
```

✓ 0.0s

- Hàm tìm điểm trung tâm phù hợp với toàn bộ các điểm:

```python
def centroid_assignation(dset, centroids, inv_cov):
    k = centroids.shape[0]
    n = dset.shape[0]
    assignation = []
    assign_errors = []

    for obs in range(n):
        #Estimate error
        all_errors = np.array([])

        for centroid in range(k):
            err = mahalanobis_distance(dset.iloc[obs, :].values,
                                       centroids.iloc[centroid, :].values,
                                       inv_cov)
            all_errors = np.append(all_errors, err)

        all_errors = np.array(all_errors)
        nearest_centroid = np.argmin(all_errors)
        nearest_centroid_error = all_errors[nearest_centroid]

        assignation.append(nearest_centroid)
        assign_errors.append(nearest_centroid_error)
    return assignation, assign_errors
```

✓ 0.0s Pytho

- Hàm tự động chạy thuật toán Mahalanobis_Kmeans, tự động điều chỉnh các điểm trung tâm sau mỗi lần lặp, dừng lại khi các điểm trung tâm không còn thay đổi quá nhiều:

```python
def Mahalanobis_KMeans(dset, k, tol=1e-4):
    cov_matrix = np.cov(dset.values, rowvar=False)
    inv_cov = np.linalg.pinv(cov_matrix)

    working_dset = dset.copy()
    err = []
    goahead = True
    j = 0

    centroids = initiate_centroids(k , dset)

    while(goahead):
        data_to_assign = working_dset.drop(columns=['centroid'], errors='ignore')

        working_dset['centroid'], j_err = centroid_assignation(data_to_assign, centroids, inv_cov)
        err.append(sum(j_err))

        centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)

        if j > 0:
            if err[j - 1] - err[j] <= tol:
                goahead = False
        j += 1

    data_to_assign = working_dset.drop(columns=['centroid'], errors='ignore')

    working_dset['centroid'], j_err = centroid_assignation(data_to_assign, centroids, inv_cov)

    centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)
    return working_dset['centroid'], j_err, centroids
```

✓ 0.0s

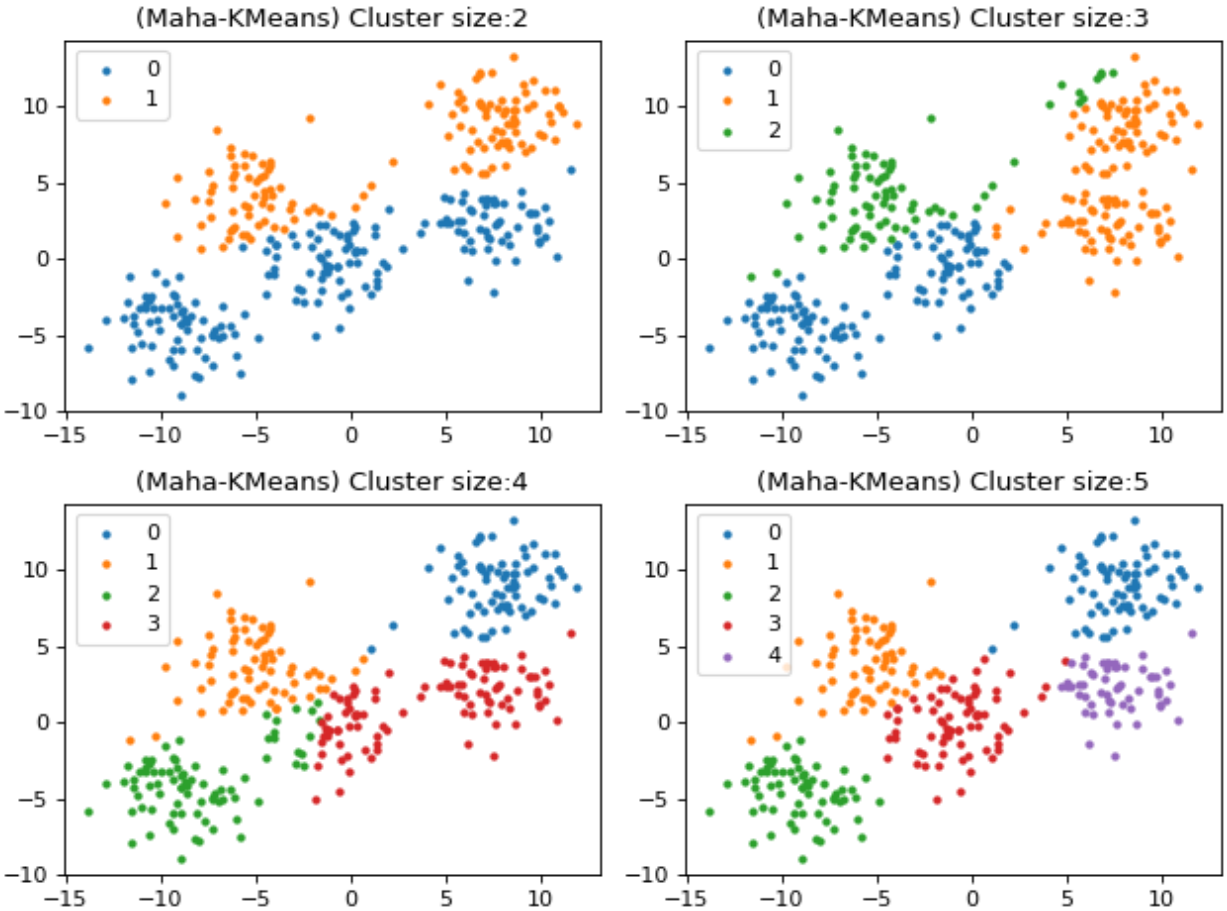
- Chạy hàm trên với bộ dữ liệu đã cho, cluster từ 2 đến 5:

```python
f = plt.figure(figsize=(8, 6), dpi=80)

for i in range(2, 6):
    pred, _, _ = Mahalanobis_KMeans(pd.DataFrame({'x':x[:,0], 'y':x[:, 1]}), i)
    df = pd.DataFrame({'x':x[:,0], 'y':x[:, 1], 'label':pred})
    groups = df.groupby('label')
    f.add_subplot(2, 2, i - 1)
    for name, group in groups:
        plt.scatter(group.x, group.y, label=name, s=8)
        plt.title("(Maha-KMeans) Cluster size:" + str(i))
        plt.legend()

plt.tight_layout()
plt.show()
```

✓ 3.0s


(Maha-KMeans) Cluster size:2
0
1
(Maha-KMeans) Cluster size:3
0
1
2
(Maha-KMeans) Cluster size:4
0
1
2
3
(Maha-KMeans) Cluster size:5
0
1
2
3
4
10
5
0
−5
−10
−15
−10
−5
0
5
10

Nhận xét về GMM, thuật toán k-means và Mahalanobis k-means:

1. Thuật toán K-means (Hard Clustering)

K-means là thuật toán phân cụm dựa trên khoảng cách Euclidean, giả định các cụm có hình dạng cầu và kích thước tương đương.

- **Ưu điểm:** Tốc độ tính toán rất nhanh, dễ cài đặt và hiệu quả với các tập dữ liệu lớn, phân tách rõ ràng.
- **Hạn chế:** * Nhạy cảm với nhiễu (outliers).
    - Chỉ hoạt động tốt với các cụm hình cầu.
    - Là dạng **Hard Clustering** (mỗi điểm chỉ thuộc về duy nhất một cụm), không thể hiện được mức độ tự tin của việc phân loại.

2. Khoảng cách Mahalanobis (Đo lường sự tương quan)

Khác với Euclidean, khoảng cách Mahalanobis tính đến phân phối (hiệp phương sai) của dữ liệu.

- **Đặc điểm:** Nó đo khoảng cách từ một điểm đến tâm của một phân phối theo đơn vị độ lệch chuẩn.
- **Ứng dụng:** Thường dùng để phát hiện điểm dị biệt (outlier detection) hoặc làm nền tảng toán học cho các thuật toán phân cụm phức tạp hơn.
- **Nhận xét:** Nó giải quyết được bài toán dữ liệu có dạng hình Elip (không đồng nhất về phương sai giữa các chiều), điều mà Euclidean thất bại.

3. Gaussian Mixture Model - GMM (Soft Clustering)

GMM là một mô hình xác suất giả định dữ liệu được tạo ra từ sự pha trộn của nhiều phân phối Gaussian.

- **Cơ chế:** Sử dụng thuật toán Expectation-Maximization (EM) để tối ưu hóa tham số. GMM thực chất là một dạng tổng quát hóa của K-means.
- **Ưu điểm nổi bật:**
    - **Soft Clustering:** Cung cấp xác suất một điểm thuộc về các cụm khác nhau (giúp xử lý các vùng dữ liệu chồng lấn).
    - **Linh hoạt:** Nhờ sử dụng ma trận hiệp phương sai (liên quan mật thiết đến khoảng cách Mahalanobis), GMM có thể thích nghi với các cụm có hình dạng Elip, kích thước và định hướng khác nhau.
- **Hạn chế:** Chi phí tính toán cao hơn K-means và có thể bị kẹt ở cực trị địa phương (local optima).